

Lab 5: Parallel Skeletons - Detailed Study Guide

This extensively expanded guide covers "Skeletons"—high-level structural patterns for parallel computing. Instead of manually creating threads and handling mutexes, you use these patterns (like templates) where the framework handles the parallelization, and you simply provide the core computation logic.

1. Data Parallel Skeletons (Collection-based)

A. Map Skeleton

- **Concept:** Applies a function $f(x)$ independently to every single element of an array. Because $f(x_1)$ has strictly zero relation to $f(x_2)$, this is considered "Embarrassingly Parallel".
- **Use Cases:** Squaring numbers, converting an image to grayscale pixel-by-pixel, parsing independent log files.
- **Implementation Example (Python):**

```
import multiprocessing

def square(x):
    return x * x

if __name__ == '__main__':
    numbers = [1, 2, 3, 4, 5, 6, 7, 8]
    # Spawns a pool of worker processes (usually matches CPU core count)
    with multiprocessing.Pool() as pool:
        # pool.map automatically chunks 'numbers' and distributes to workers
        result = pool.map(square, numbers)
    print(result) # [1, 4, 9, 16, 25, 36, 49, 64]
```

B. Reduce Skeleton

- **Concept:** Combines all elements into a single scalar value using a **binary associative operator** (like $+$, $*$, $\min$, $\max$).
- **Why Associativity Matters:** Associativity guarantees that $(A+B)+C = A+(B+C)$. This mathematical property allows parallel execution because we don't have to evaluate strictly left-to-right. We can build a **Reduction Tree**.
- **How the tree works:** Given $[2, 4, 6, 8]$:
 - Thread 1 computes $2+4 = 6$
 - Thread 2 computes $6+8 = 14$
 - Thread 1 computes final $6+14 = 20$
- **Use Cases:** Calculating total sum, finding the maximum element in a massive dataset, counting word frequencies.

C. Scan Skeleton (Prefix Sum)

- **Concept:** Computes running totals (partial reductions).
 - Input: [1, 2, 3, 4]
 - Output: [1, 3, 6, 10]
- **Implementation Strategy:** *The Hillis-Steele Algorithm*. Instead of thread 1 passing a sum sequentially to thread 2, we do this in steps (hops):
 - **Step 1 (hop=1):** Every element adds the value 1 slot to its left.
 - **Step 2 (hop=2):** Every element adds the value 2 slots to its left.
 - **Step 3 (hop=4):** Every element adds the value 4 slots to its left. Because the hop size doubles every step, this completes in $O(\log n)$ steps rather than sequential $O(n)$ steps.

```
# Pseudo-code for Hillis-Steele (assuming thread-safe temp array)
for hop in [1, 2, 4, 8, 16]: # Powers of 2 up to N
    # Everything in this loop can happen in parallel
    for i in range(hop, n):
        res[i] = temp[i] + temp[i - hop]
    # synchronize here
```

2. Task Parallel Skeletons (Workload-based)

A. Farm (Master-Worker)

- **Concept:** A Master process parses incoming data and pushes individual "Tasks" into a shared thread-safe Queue. A fleet of Worker processes pull tasks from the queue, process them, and put results in a Results Queue.
- **Key Benefit: Dynamic Load Balancing.** If Task A takes 1 hour and Task B takes 1 second, the worker who finishes Task B just immediately grabs Task C from the queue. No worker sits completely idle (unlike static chunking).

B. Divide and Conquer (D&C)

- **Concept:** Recursively halves the problem size until it hits a base case. It processes the halves concurrently, and merges the results back up the tree.
- **Use Cases:** Parallel Merge Sort, Quick Sort.
- **Implementation Example (Python Parallel Merge Sort):**

```
import concurrent.futures

def merge_sort_parallel(arr):
    if len(arr) <= 1:
        return arr
    if len(arr) <= 1000: # Base case threshold to avoid thread overhead
        return sorted(arr)

    mid = len(arr) // 2
    # Dispatch left and right halves to execute in parallel
```

```

with concurrent.futures.ThreadPoolExecutor(max_workers=2) as executor:
    future_left = executor.submit(merge_sort_parallel, arr[:mid])
    future_right = executor.submit(merge_sort_parallel, arr[mid:])

    # Wait for both halves to finish sorting
    left_sorted = future_left.result()
    right_sorted = future_right.result()

    return merge(left_sorted, right_sorted) # Standard sequential merge

```

C. Wavefront

- **Concept:** Processing 2D grids along the *diagonals*. This happens in algorithms like Longest Common Subsequence (LCS) or heat diffusion mapped over 2D arrays.
- **The Dependency Problem:** Cell (i, j) relies on answers from $(i-1, j)$, $(i, j-1)$, and $(i-1, j-1)$. You cannot calculate row-by-row in parallel, nor column-by-column.
- **Implementation Example (Python LCS DP Matrix):** Instead of typical row/col nested loops, we iterate over the diagonals. All cells within diagonal k can be computed completely in parallel.

```

import multiprocessing as mp

def compute_cell(args):
    i, j, str1, str2, dp_matrix = args
    if str1[i-1] == str2[j-1]:
        return (i, j, dp_matrix[i-1][j-1] + 1)
    else:
        return (i, j, max(dp_matrix[i-1][j], dp_matrix[i][j-1]))

def lcs_wavefront(str1, str2):
    n, m = len(str1), len(str2)
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    with mp.Pool() as pool:
        # Iterate through diagonals k
        for k in range(2, n + m + 1):
            # Gather all valid (i,j) coordinates for this specific diagonal
            tasks = []
            for i in range(1, n + 1):
                j = k - i
                if 1 <= j <= m:
                    tasks.append((i, j, str1, str2, dp))

            # Submit the entire diagonal to the worker pool in PARALLEL
            results = pool.map(compute_cell, tasks)

            # Write computed answers back to matrix before moving to next
            diagonal
            for r_i, r_j, val in results:
                dp[r_i][r_j] = val

```

```
return dp[n][m]
```

3. Exam Cheat Sheet: Choosing and Adapting Skeletons

If the question says...	Which Skeleton to Use?	What to write/adapt
"Apply a function to every item independently"	Map Skeleton	Python: <code>with mp.Pool() as p: p.map(func, array)</code>
"Tasks arrive continuously and take unpredictable times"	Farm (Master-Worker)	Use shared concurrent Queues (<code>mp.Queue()</code>). Master puts tasks, Workers loop <code>queue.get()</code> . Send a "Poison Pill" (e.g. <code>None</code>) to stop workers.
"Compute a final aggregate value (sum, min, max)"	Reduce Skeleton	Ensure your operator is associative! (e.g. Addition). In OpenMP, simply use <code>reduction(+:var)</code> .
"Compute running totals / prefix sums"	Scan Skeleton	State that you will use Hillis-Steele ($O(\log n)$). Write a loop where distance/hop doubles each step.
"Sort an array in parallel"	Divide and Conquer	Recursively split half. Async dispatch the left and right halves. Wait for both (<code>future.result()</code>). Then sequentially merge.
"Update a 2D grid where cell (i,j) depends on $(i-1,j)$ and $(i,j-1)$ "	Wavefront Pattern	You CANNOT use basic Map. Iterate over diagonals k . Inner loop finds (i,j) where $i+j=k$. Process all cells in diagonal k in parallel.

4. Multi-Layer Exam Preparation: Solved Mock Questions

Below are high-probability exam strings emphasizing skeleton identification.

Question A: The Web Server Pool Structure

Problem Statement: A web server receives HTTP requests containing highly varying computational loads (some just ask for the homepage, others ask the server to generate a 3D rendering). You are tasked to design a scalable parallel backend.

1. Which skeleton should you use?
2. Why is a standard Map skeleton a poor choice here?

Step-by-step Solution:

1. **Skeleton Choice:** You must use the **Farm (Master-Worker)** skeleton. A Master thread will listen on the network port and place incoming HTTP requests into a shared synchronized queue. A pool of worker threads will continually pop requests off the queue and process them.

2. **Why Map Fails:** Map generally divides an array of size N statically among W workers (each gets N/W items). If Worker 1 gets all the "3D rendering" requests by chance, and Worker 2 gets all the "homepage" requests, Worker 2 will finish in 5 milliseconds and sit completely idle while Worker 1 grinds for an hour. Farm provides **Dynamic Load Balancing**, preventing idle cores.

Question B: Associative Operator Identification

Problem Statement: You have an array of integers. You want to compute the running sum (prefix sum) of the array and simultaneously find the average of the whole array.

1. Which skeleton computes the running sum?
2. Why can't you write a standard **Reduce** skeleton passing an **average()** function as the binary operator? How do you actually compute the average using skeletons?

Step-by-step Solution:

1. **Skeleton for Running Sum:** The **Scan** skeleton. Specifically, the Hillis-Steele or Blelloch scan algorithms.
2. **Why **average()** fails in Reduce:** A Reduce skeleton *requires* the binary operator to be strictly associative. Averages are NOT associative. $\text{Avg}(\text{Avg}(10, 20), 30) \neq \text{Avg}(10, \text{Avg}(20, 30))$. The left side is $\text{Avg}(15, 30) = 22.5$. The right side is $\text{Avg}(10, 25) = 17.5$.
3. **How to compute average correctly:** Use a **Reduce** skeleton specifying **Addition** as the operator to find the Total Sum. Have another variable track the array length N . Then do a sequential division **Total Sum / N** at the very end.

Question C: Wavefront Diagonalization

Problem Statement: Write pseudo-code explaining how to iterate a 3x3 matrix using the Wavefront pattern. What is the formula to find elements on the k^{th} diagonal?

Step-by-step Solution:

1. A matrix of size $N \times M$ has $N + M - 1$ diagonals.
2. For a 3x3 matrix, there are $3+3-1 = 5$ diagonals.
3. The coordinates (i, j) of cells on diagonal k always satisfy the property $i + j == k$. (Assuming 0-indexed).
4. Pseudo-code:

```
N = 3, M = 3
# k goes from 0 to 4 (representing the 5 diagonals)
for k in range(N + M - 1):
    # EVERYTHING INSIDE THIS LOOP EXECUTES IN PARALLEL
    for i in range(N):
        j = k - i
        if (j >= 0 and j < M): # Make sure the cell actually exists in the
            grid
                calculate_cell(i, j)
    # MUST SYNCHRONIZE HERE before moving to the next diagonal (k+1)
```

5. **Explanation:** For $k=2$, the loop finds valid tuples $(i=0, j=2)$, $(i=1, j=1)$, and $(i=2, j=0)$. These three cells are computed perfectly in parallel.